ENGINEERING AI SYSTEMS

Agentic Observability: From Blind Spots to Multiagent traces

Instrumenting Microsoft Agent Framework with OpenTelemetry, Grafana, and Azure.

The Failure Mode

$80.00

Burned in a single infinite agent loop.

Root cause unknown.

The Traditional Baseline

Calibration: How we observe standard software today.



Logs

Discrete timestamped events.



Metrics

Aggregated numeric trends.

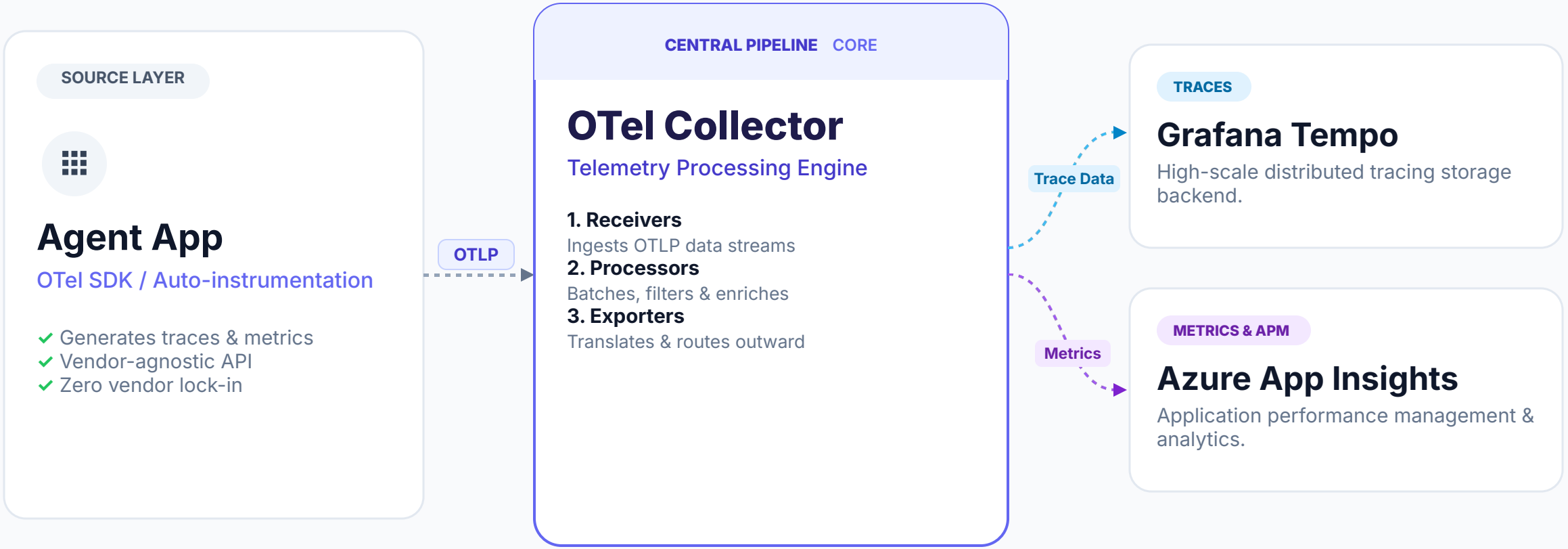


Traces

Causality across distributed steps.

OpenTelemetry Architecture

The vendor-neutral standard for telemetry routing and processing.



Why Agents Break Observability

Key architectural differences comparing traditional APIs vs autonomous AI agents

Traditional APIs

EXECUTION PATTERN

Deterministic, linear execution.

TRACE HIERARCHY

Static trace hierarchies.

COST MODEL

Cost scales with request volume.

Agentic Workflows

REASONING PATTERN

Non-deterministic reasoning loops.

TOOL CALLING

Deep, recursive tool-calling trees.

DYNAMIC COST

Cost scales dynamically via tokens.

GenAI Semantic Conventions

Standardizing attributes across all model providers.

ATTRIBUTE KEY	VALUE EXAMPLE	PURPOSE
`gen_ai.provider.name`	`"openai", "anthropic"`	Provider identity.
`gen_ai.request.model`	`"gpt-4o", "claude-3-opus"`	Routing and cost analysis.
`gen_ai.usage.input_tokens`	`1450`	Context window tracking.
`gen_ai.system_instructions`	`"You are a helpful assistant ... "`	Debugging hallucinations.

https://github.com/open-telemetry/semantic-conventions-genai

Microsoft Agent Framework

OpenTelemetry integration.

Zero-Config Instrumentation

MAF wraps LLM calls and tool executions automatically.

Seamless Integration

Single method setup for full OpenTelemetry span coverage.

Full Context

Captures prompts, completions, and recursive tool calls in real time.

SECURITY NOTICE

The Sensitive Data Tradeoff

Capturing prompts enables debugging, but introduces PII/compliance risks into traces. Ensure sensitive data flags are evaluated before deployment.

`ENABLE_SENSITIVE_DATA=true`

agent_observability.py

```python
from agent_framework.observability import configure_otel_providers
from agent_framework.openai import OpenAIChatClient

# 1. Initialize standard OTel providers

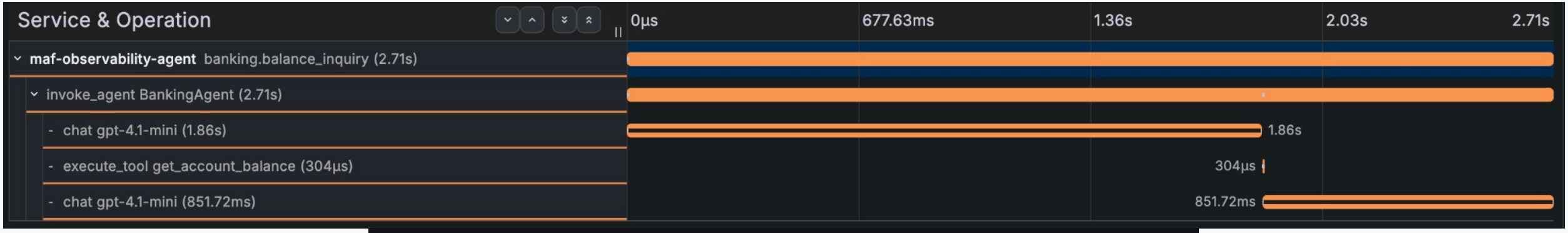
# 1. Reads OTEL_EXPORTER_OTLP_* env vars
# OTEL_EXPORTER_OTLP_ENDPOINT=your-endpoint
# OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf

configure_otel_providers()

# 2. Agent runs normally. Spans emitted automatically.

agent = Agent(
      client=OpenAIChatClient(),
      tools=get_account_balance,
      name="BankingAgent",
      id="banking-agent",
)

await agent.run("What's my balance on account ACC-1002?")
```



Span attributes

Network 1

Other 16

gen_ai.output.messages	"content": "The balance on your account ACC-1002 is $187.55." }], "finish_reason": "stop" }]
gen_ai.provider.name	azure.ai.openai
gen_ai.request.choice.count	1
gen_ai.request.model	gpt-4.1-mini
gen_ai.response.finish_reasons	["stop"]
gen_ai.response.id	resp_0d78f747155049db006abb2323d3e48197b820cbbaaf965f10
gen_ai.response.model	gpt-4.1-mini-2025-04-14
gen_ai.system_instructions	[{ "type": "text", "content": "You are a helpful banking assistant. You can look up account balances via the get_account_balance tool. Only look up accounts the user explicitly gives you an ID for."

Multiagent Observability

Agent Communication

HTTP + A2A. Traceparent attribute

MAF (configure_otel_providers):
Agent Spans: invoke_agent, chat, execute_tool

HTTPXClientInstrumentor:
Injects traceparent (OUT)

FastAPIInstrumentor:
Extracts traceparent (IN)



otel_config.py

```python
def setup_observability(service_name: str) -> None:
    configure_otel_providers(service_name=service_name)
    HTTPXClientInstrumentor().instrument()                # OUT
    LoggingInstrumentor().instrument(set_logging_format=True,
log_level=INFO)

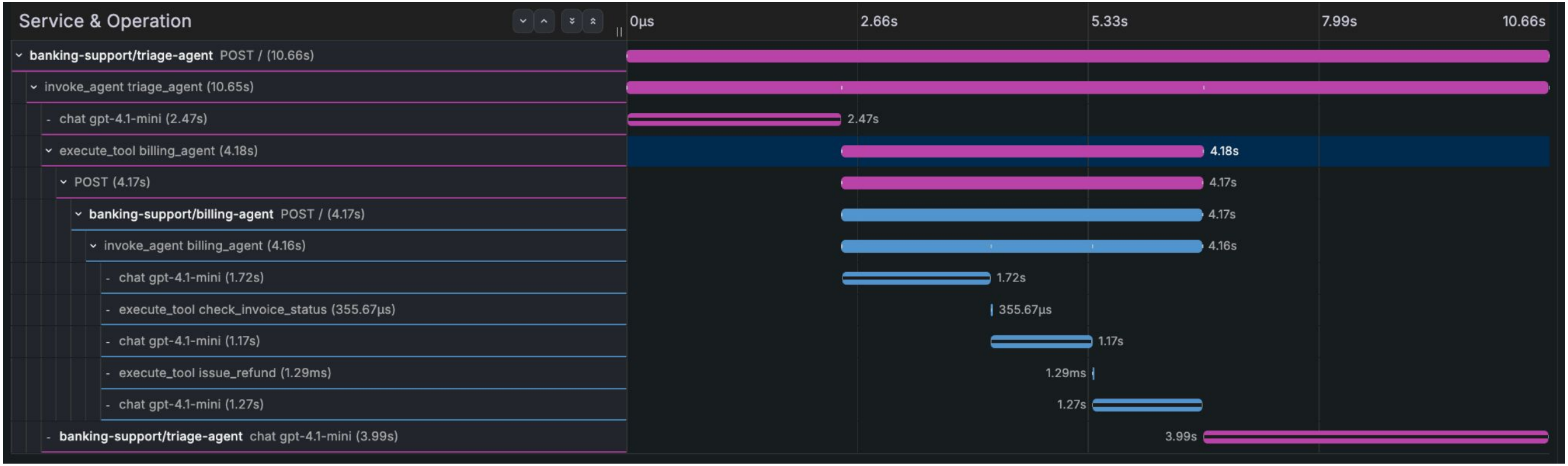
def instrument_app(app: FastAPI) -> None:
    FastAPIInstrumentor.instrument_app(app)               # IN
```

.env

```
OTEL_EXPORTER_OTLP_ENDPOINT=https://otlp-gateway-<region>.grafana.net/otlp
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
OTEL_PROPAGATORS=tracecontext,baggage
OTEL_RESOURCE_ATTRIBUTES=service.namespace=banking-support,deployment.envir
onment=local

OTEL_SERVICE_NAME=billing-agent # ← the only per-agent line
```

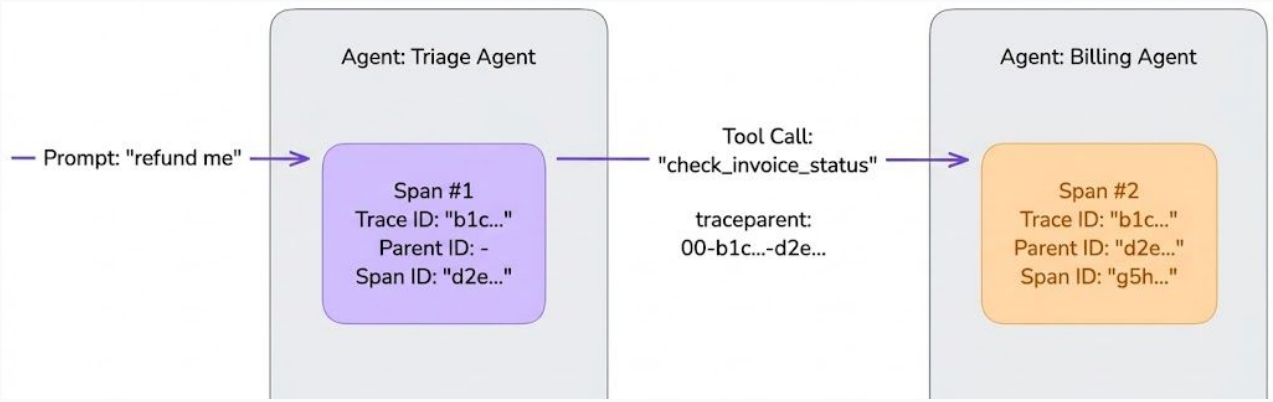
Prompt: I was double charged on account ACC-4471. Check my invoice and refund me 42.50 USD.



Final Response: I checked the latest invoice for account ACC-4471, and it is currently unpaid with a due date in 3 days. I have also processed a refund of $42.50 as requested for the double charge. The refund has been issued to the account. If you need further assistance, please let me know.

Multiagent Observability: traceparent is per-hop not per-trace

- **`span-id`**: Unique value per trace. trace identifier
- **`trace-id`**: Makes it a single, unified trace.
- **`parent-span-id`**: Makes it a tree structure.
  - It is the span ID of whoever made that particular call.

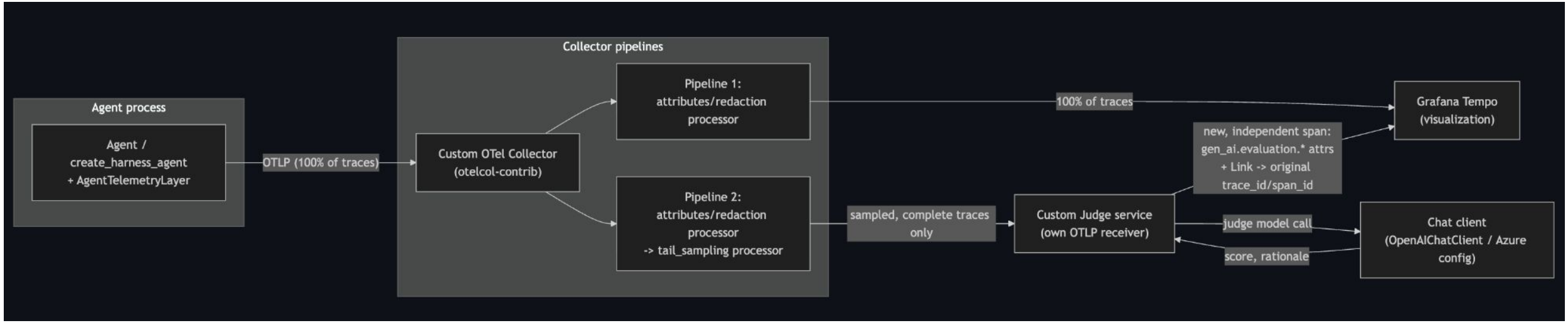



```
traceparent:  00 - aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa - c47f079564823be3 - 01
              ↑          ↑                              ↑                ↑
          version    trace-id                    parent-span-id     flags
                  CONSTANT across the trace     CHANGES every hop  (01 = sampled)
```


Dashboards › Agentic Observability
Search...
Invite
Last 15 minutes
Refresh
Edit
Tokens/s
106
input
21.8
output
Tokens/s by model
gpt-4.1
61.8
gpt-4.1-mini
66.5
Model latency p95 by agent + model
16
14
12
10
8
6
4
2
0
20:00
20:05
20:10
{gen_ai_request_model="gpt-4.1", job="banking-support/billing-agent-gpt41"}
{gen_ai_request_model="gpt-4.1", job="banking-support/tech-agent-gpt41"}
{gen_ai_request_model="gpt-4.1", job="banking-support/triage-agent-gpt41"}
{gen_ai_request_model="gpt-4.1-mini", job="banking-support/billing-agent-gpt41-mini"}
Model calls/sec per agent
0.5
0.4
0.3
0.2
0.1
0
20:00
20:05
20:10
banking-support/billing-agent-gpt41
banking-support/billing-agent-gpt41-mini
banking-support/tech-agent-gpt41
banking-support/tech-agent-gpt41-mini
banking-support/triage-agent-gpt41
banking-support/triage-agent-gpt41-mini
Tokens per user request
1152
Agent p95 end-to-end latency (s)
banking-support/billing-agent-gpt41
4.88
banking-support/billing-agent-gpt41-mini
10
banking-support/tech-agent-gpt41
7.21
banking-support/tech-agent-gpt41-mini
7
banking-support/triage-agent-gpt41
10
banking-support/triage-agent-gpt41-mini
10
Agent request rate
0.0217
banking-support/billing-agent-gpt41
0.0254
banking-support/billing-agent-gpt41-mini
0.0296
banking-support/tech-agent-gpt41
0.0338
banking-support/tech-agent-gpt41-mini
0.0551
banking-support/triage-agent-gpt41
0.0563
banking-support/triage-agent-gpt41-mini

Bonus: Continuous Evaluation

- **Collector - Pipeline 1:** 100% traces to Grafana/Tempo
- **Collector - Pipeline 2:** tail sampling processor, sampling traces to Judge Service
- **Judge Service:** evaluates sampled traces. Sends `gen_ai_evaluation*` traces to Grafana for agent explainability



One Trace, Two Backends

Proving vendor neutrality by splitting the OTel export stream.

Grafana Tempo

Destination A: OTLP Span Exporter

Azure App Insights

Destination B: Azure Native Exporter

otel_config.py

```
# Inside OTel Configuration:
provider = TracerProvider()

# Destination A: Grafana Cloud via OTLP
grafana = OTLPSpanExporter(
endpoint="[link removed]"
)
provider.add_span_processor( BatchSpanProcessor(grafana) )

# Destination B: Azure native exporter
azure = AzureMonitorTraceExporter(
connection_string=os.getenv("APP_INSIGHTS")
)
provider.add_span_processor( BatchSpanProcessor(azure) )
```



github.com/gmezan/agent-observability